Aula 3

Escola Politécnica

Arquitetura de Sistemas

Profª Drª Mariane Gavioli Bergamini Fernandes

Conversa Inicial

- Os objetivos da aula serão: identificar as melhorias e aplicar métodos eficientes para a reconstrução das arquiteturas; Analisar e avaliar os riscos relacionados as mudanças; compreender algumas estratégias de versionamento.

- Durante nossa aula será explorado 5 temas:
- Reconstrução de processos de arquitetura
- Extração de dados, construção de dados e análise de arquitetura (violações encontradas)
- Estratégias de Versionamento
- Risco de mudança, desempenho, tolerância ao erro e quando usar
- Replicação de código e padrões de design para reuso

RECONSTRUÇÃO DE PROCESSOS DE ARQUITETURA

Modificações de arquiteturas para projetos existentes

- A Análise de conformidade avalia a correspondência entre a arquitetura de um sistema de software e sua documentação, auxiliando na atualização desta.
- A engenharia reversa está ganhando cada vez mais importância para a manutenção e evolução de sistemas de software.

Engenharia Reversa

- Obter representações do sistema em um nível mais alto de abstração e identificar os componentes fundamentais do sistema analisado e recuperando suas estruturas constituintes.

- Código fonte;
- Informações de usuários e/ou analista;
- Documentação existente (manual de usuário, manual de sistema, modelos, fluxogramas, etc.)

Processos de reconstrução

- O que se deve olhar?
- Arquitetura atual;
- Proposito da mudança;
- Escolha da nova arquitetura;
- Implementação da arquitetura;
- Deploy e monitoramento.

EXTRAÇÃO DE DADOS, CONSTRUÇÃO DE DADOS E ANÁLISE DE ARQUITETURA (VIOLAÇÕES ENCONTRADAS)

Data Warehouse (DW)

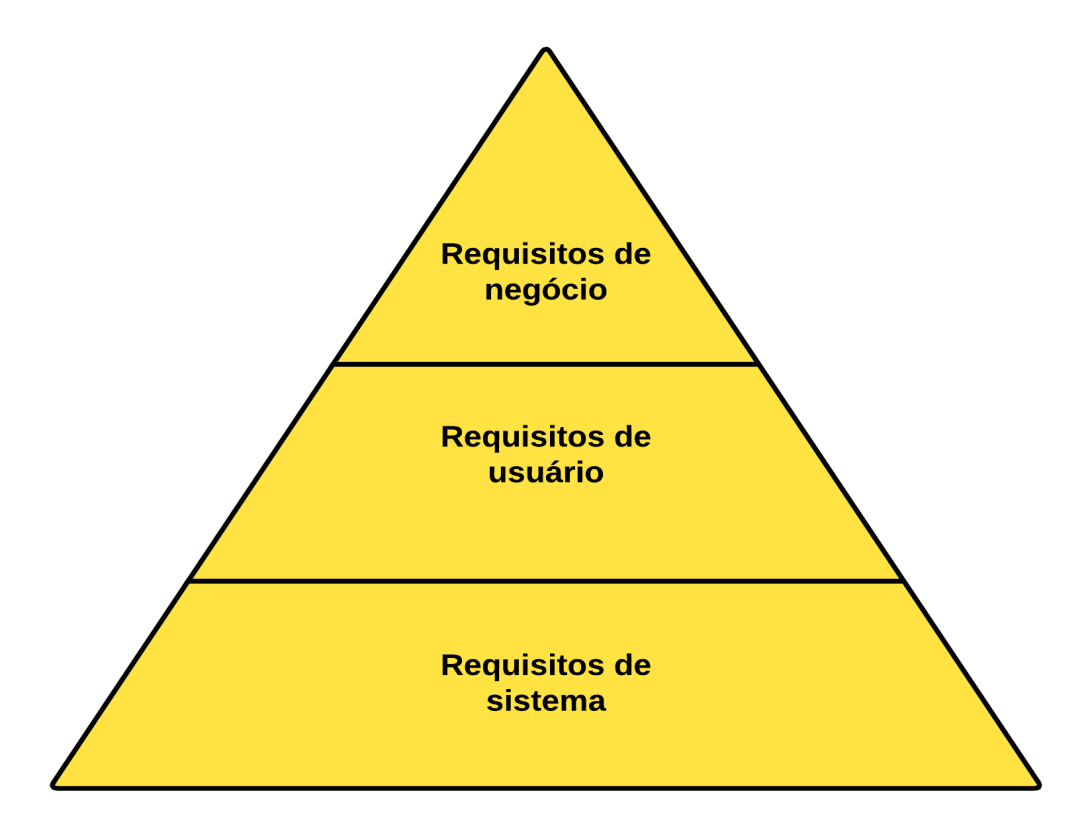
- "Data Warehouse está relacionado a ideia de uma base de dados distintos níveis de granularidade. Essa base pode ser obtidos a partir de sistemas transacionais que servem de fonte de dados, armazenando os mesmos em diferentes gruas de conhecimento. " (Barbieri, 2001)

ETC ou ETL(Extração, Transformação e Carregamento)

- Extração: responsável pela extração dos dados das fontes informacionais.
- Limpeza e Transformação.
- Carregamento: os dados extraídos e transformados são inseridos nas estruturas dimensionais.

Componentes do DW

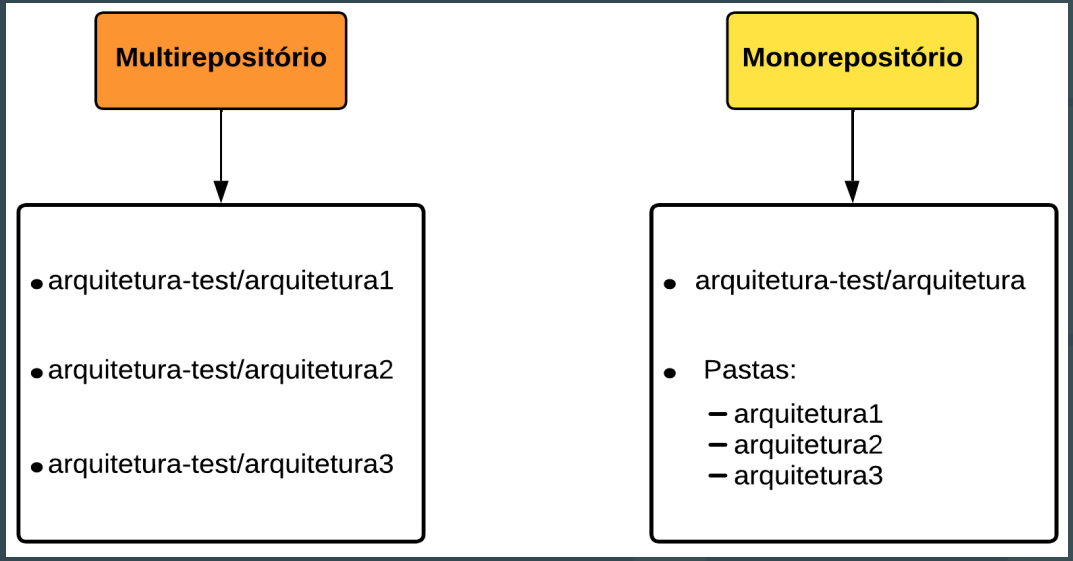
- Fontes informacionais – podendo ser internas ou externas.
- Software para Extração, Transformação e Carregamento – ferramentas que permitem extrair os dados das diversas fontes informacionais, transformá-los e carregá-los no DW (data marts);
- Repositórios – constituída por diferentes elementos, pelo DW, ou metadados;
- Ferramentas de análise – permitem aos utilizadores acederem à informação armazenada nos repositórios.


Requisitos de negócio
Requisitos de usuário
Requisitos de sistema

ESTRATÉGIAS DE VERSIONAMENTO

Versionamento para arquiteturas de sistemas

- Controle de versão;
- Integração contínua;
- Estratégias de *branching;*
- Desenvolvimento contínuo.



Estrátegias

- Tronco: os envolvidos confirmam as alterações na *main* ramificação várias vezes ao dia.
- Fluxo: ramificações de recursos de curta duração que mesclam à ramificação principal.

- ***Gitflow*: Após a aprovação, é mesclado as feature(recurso) ramificações em uma ramificação de integração (ramificação de desenvoler).**

Branch permanente

- Principal (main): o que está pronto para produção;
- Desenvolver (develop): recursos que são implementados, mas não passaram por um teste final.

Branch temporária

- Ramificação de recursos: implementar um novo recurso.
- Ramificação de lançamento: prepara uma nova versão para ser aprovada.
- Ramificação Hotfix: corrige bugs/erros críticos detectados na produção.

RISCO DE MUDANÇA, DESEMPENHO, TOLERÂNCIA AO ERRO E QUANDO USAR

Mudanças e riscos

- A arquitetura precisa evoluir para acomodar mudanças nos requisitos, novas tecnologias e melhorias de desempenho.

- As mudanças precisam ser pensadas e repensadas, pois podem trazer riscos aos demais subsistemas.
- A complexidade da arquitetura, a qualidade do código e a documentação já existente influenciam bruscamente.

- Para um sistema e-commerce: a decisão de migrar para microsserviços visa melhorar a escalabilidade, a manutenibilidade e a capacidade de inovação.
- Quais são os possíveis riscos?

- Risco de desempenho e latência;
- Risco de complexidade;
- Risco de consistência de dados;
- Risco de implementação e teste;
- Risco de custo;
- Risco de integração.

- Sistemas com requisitos em constante mudança podem se beneficiar de arquiteturas mais flexíveis e modulares.
- Sistemas com requisitos de desempenho exigem uma arquitetura para otimizar escalabilidade e eficiência.
- Sistemas críticos requerem arquiteturas que priorizem a tolerância a erros e a alta disponibilidade.

REPLICAÇÃO DE CÓDIGO E PADRÕES DE DESIGN PARA REUSO

Refatoração de códigos

- Vamos imaginar que temos uma função para calcular o preço final de um produto, considerando neste sistema desconto e impostos.

```python
def calcular_preco_final_carrinho(produtos):
  """Calcula o preço final dos produtos no carrinho."""
  preco_total = 0
  for produto in produtos:
    preco_unitario = produto['preco']
    desconto = produto.get('desconto', 0)  # Desconto padrão 0 se não houver
    imposto = produto.get('imposto', 0.1) # Imposto padrão de 10% se não houver
    preco_total += preco_unitario * (1 - desconto) * (1 + imposto)
  return preco_total


def calcular_preco_final_detalhes(produto):
  """Calcula o preço final na página de detalhes do produto."""
  preco_unitario = produto['preco']
  desconto = produto.get('desconto', 0)
  imposto = produto.get('imposto', 0.1)
  preco_final = preco_unitario * (1 - desconto) * (1 + imposto)
  return preco_final
```

Com fatoração:

```python
def calcular_preco_final(preco_unitario, desconto=0, imposto=0.1):
  """Calcula o preço final de um produto."""
  return preco_unitario * (1 - desconto) * (1 + imposto)

def calcular_preco_final_carrinho(produtos):
  """Calcula o preço final dos produtos no carrinho."""
  preco_total = 0
  for produto in produtos:
    preco_total += calcular_preco_final(produto['preco'], produto.get('desconto', 0), produto.get('imposto', 0.1))
  return preco_total

def calcular_preco_final_detalhes(produto):
  """Calcula o preço final na página de detalhes do produto."""
  return calcular_preco_final(produto['preco'], produto.get('desconto', 0), produto.get('imposto', 0.1))
```

Padrões de design

- Fábrica Abstrata (*Abstract Factory*)
- Construtor (*Builder*)
- Metódo de fatoração (*Factory Method*)
- Protótipo (*Prototype*)
- *Singleton*
- Adpatar *(Adpater)*
- Ponte *(Bridge)*

Fechar